# Record-Klassen

Carsten Gips (HSBI)

**TL;DR**

Häufig schreibt man relativ viel *Boiler Plate Code*, um einfach ein paar Daten plus den Konstruktor und die Zugriffsmethoden zu kapseln. Und selbst wenn die IDE dies zum Teil abnehmen kann - lesen muss man diesen Overhead trotzdem noch.

Für den Fall von Klassen mit `final` Attributen wurden in Java14 die **Record-Klassen** eingeführt. Statt dem Schlüsselwort `class` wird das neue Schlüsselwort `record` verwendet. Nach dem Klassennamen kommen in runden Klammern die "Komponenten" - eine Auflistung der Parameter für den Standardkonstruktor (Typ, Name). Daraus wird automatisch ein "kanonischer Konstruktor" mit exakt diesen Parametern generiert. Es werden zusätzlich `private final` Attribute generiert für jede Komponente, und diese werden durch den kanonischen Konstruktor gesetzt. Außerdem wird für jedes Attribut automatisch ein Getter mit dem Namen des Attributs generiert (also ohne den Präfix "get").

Beispiel:

```
public record StudiR(String name, int credits) {}
```

Der Konstruktor und die Getter können überschrieben werden, es können auch eigene Methoden definiert werden (eigene Konstruktoren *müssen* den kanonischen Konstruktor aufrufen). Es gibt außer den über die Komponenten definierten Attribute keine weiteren Attribute. Da eine Record-Klasse intern von `java.lang.Record` ableitet, kann eine Record-Klasse nicht von weiteren Klassen ableiten (erben). Man kann aber beliebig viele Interfaces implementieren. Record-Klassen sind implizit final, d.h. man nicht von Record-Klassen erben.

**Videos**

- VL Record-Klassen
- Demo Record-Klassen

**Unterlagen**

- PDF-Version

---

# 1 Motivation; Klasse Studi

```
public class Studi {
    private final String name;
    private final int credits;

    public Studi(String name, int credits) {
        this.name = name;
        this.credits = credits;
    }

    public String getName() {
        return name;
    }

    public int getCredits() {
        return credits;
    }
}
```

## 2 Klasse Studi als Record

```
public record StudiR(String name, int credits) {}
```

- Immutable Klasse mit Feldern `String name` und `int credits`
  => "`(String name, int credits)`" werden "Komponenten" des Records genannt
- Standardkonstruktor setzt diese Felder ("Kanonischer Konstruktor")
- Getter für beide Felder:

  ```
  public String name() { return this.name; }
  public int credits() { return this.credits; }
  ```

Record-Klassen wurden in Java14 eingeführt und werden immer wieder in neuen Releases erweitert/ergänzt.

Der kanonische Konstruktor hat das Aussehen wie die Record-Deklaration, im Beispiel also `public StudiR(String name, int credits)`. Dabei werden die Komponenten über eine Kopie der Werte initialisiert.

Für die Komponenten werden automatisch private Attribute mit dem selben Namen angelegt.

Für die Komponenten werden automatisch Getter angelegt. Achtung: Die Namen entsprechen denen der Komponenten, es fehlt also der übliche "get"-Präfix!

## 3 Eigenschaften und Einschränkungen von Record-Klassen

- Records erweitern implizit die Klasse `java.lang.Record`:
  Keine andere Klassen mehr erweiterbar! (Interfaces kein Problem)
- Record-Klassen sind implizit final
- Keine weiteren (Instanz-) Attribute definierbar (nur die Komponenten)
- Keine Setter definierbar für die Komponenten: Attribute sind final
- Statische Attribute mit Initialisierung erlaubt

## 4 Records: Prüfungen im Konstruktor

Der Konstruktor ist erweiterbar:

```
public record StudiS(String name, int credits) {
    public StudiS(String name, int credits) {
        if (name == null) { throw new IllegalArgumentException("Name cannot be
            null!"); }
        else { this.name = name; }

        if (credits < 0) { this.credits = 0; }
        else { this.credits = credits; }
```

```
    }
}
```

In dieser Form muss man die Attribute selbst setzen.

Alternativ kann man die “kompakte” Form nutzen:

```
public record StudiT(String name, int credits) {
    public StudiT {
        if (name == null) { throw new IllegalArgumentException("Name cannot be
            null!"); }

        if (credits < 0) { credits = 0; }
    }
}
```

In der kompakten Form kann man nur die Werte der Parameter des Konstruktors ändern. Das Setzen der Attribute ergänzt der Compiler nach dem eigenen Code.

Es sind weitere Konstruktoren definierbar, diese *müssen* den kanonischen Konstruktor aufrufen:

```
public StudiT() {
    this("", 42);
}
```

## 5 Getter und Methoden

Getter werden vom Compiler automatisch generiert. Dabei entsprechen die Methoden-Namen den Namen der Attribute:

```
public record StudiR(String name, int credits) {}

public static void main(String... args) {
    StudiR r = new StudiR("Sabine", 75);

    int x = r.credits();
    String y = r.name();
}
```

Getter überschreibbar und man kann weitere Methoden definieren:

```
public record StudiT(String name, int credits) {
    public int credits() { return credits + 42; }
    public void wuppie() { System.out.println("WUPPIE"); }
}
```

Die Komponenten/Attribute sind aber `final` und können nicht über Methoden geändert werden!

## 6 Beispiel aus den Challenges

In den Challenges zum Thema Optional gibt es die Klasse `Katze` in den Vorgaben.

Die Katze wurde zunächst “klassisch” modelliert: Es gibt drei Eigenschaften `name`, `gewicht`und `lieblingsBox`. Ein Konstruktor setzt diese Felder und es gibt drei Getter für die einzelnen Eigenschaften. Das braucht 18 Zeilen Code (ohne Kommentare Leerzeilen). Zudem erzeugt der Boilerplate-Code relativ viel “visual noise”, so dass der eigentliche Kern der Klasse schwerer zu erkennen ist.

In einem Refactoring wurde diese Klasse durch eine äquivalente Record-Klasse ersetzt, die nur noch 2 Zeilen Code (je nach Code-Style auch nur 1 Zeile) benötigt. Gleichzeitig wurde die Les- und Wartbarkeit deutlich verbessert.

33 markdown/modern-java/src/challenges/optional/Katze.java Viewed

```diff
@@ -1,30 +1,9 @@
 package challenges.optional;

- public class Katze {
-     private String name;
-     private float gewicht;
-     private Box lieblingsBox;
-
-     /**
-      * @param name Name der Katze
-      * @param gewicht Gewicht der Katze in kg
-      * @param lieblingsBox Lieblingsbox der Katze
-      */
-     public Katze(String name, float gewicht, Box lieblingsBox) {
-         this.name = name;
-         this.gewicht = gewicht;
-         this.lieblingsBox = lieblingsBox;
-     }
-
-     public String name() {
-         return name;
-     }
-
-     public float gewicht() {
-         return gewicht;
-     }
-
-     public Box lieblingsBox() {
-         return lieblingsBox;
-     }
+ /**
+  * @param name         Name der Katze
+  * @param gewicht      Gewicht der Katze in kg
+  * @param lieblingsBox Lieblingsbox der Katze
+  */
+ public record Katze(String name, float gewicht, Box lieblingsBox) {
 }
```

## 7 Wrap-Up

- Records sind immutable Klassen:
  - `final` Attribute (entsprechend den Komponenten)
  - Kanonischer Konstruktor
  - Automatische Getter (Namen wie Komponenten)
- Konstruktoren und Methoden können ergänzt/überschrieben werden
- Keine Vererbung von Klassen möglich (kein `extends`)

Schöne Doku: “Using Record to Model Immutable Data”.

# 8 Zum Nachlesen

- Oracle Corporation (2025)

---

**Lernziele**

- k2: Ich verstehe, dass Record-Klassen implizit final sind
- k2: Ich weiss, dass Record-Klassen einen kanonischen Konstruktor haben
- k2: Ich verstehe, dass die Attribute in Record-Klassen implizit final sind und automatisch angelegt und über den Konstruktor gesetzt werden
- k2: Ich weiss, dass die Getter in Record-Klassen so benannt sind wie die Namen der Komponenten, also keinen Präfix 'get' haben
- k2: Ich weiss, dass der kanonische Konstruktor ergänzt werden kann
- k2: Ich weiss, dass weitere Methoden definiert werden können
- k2: Ich verstehe, dass Record-Klassen nicht von anderen Klassen erben können, aber Interfaces implementieren können
- k3: Ich kann Record-Klassen praktisch einsetzen

**Challenges**

Betrachen Sie den folgenden Code:

```
public interface Person {
    String getName();
    Date getBirthday();
}

public class Student implements Person {
    private static final SimpleDateFormat DATE_FORMAT = new
        SimpleDateFormat("dd.MM.yyyy");

    private final String name;
    private final Date birthday;

    public Student(String name, String birthday) throws ParseException {
        this.name = name;
        this.birthday = DATE_FORMAT.parse(birthday);
    }

    public String getName() { return name; }
    public Date getBirthday() { return birthday; }
}
```

Schreiben Sie die Klasse `Student` in eine Record-Klasse um. Was müssen Sie zusätzlich noch tun, damit die aktuelle API erhalten bleibt?

---

**Quellen**

Oracle Corporation. 2025. „Learn Java“. 2025. https://dev.java/learn/.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** e6faa17 (lecture: improve learning outcomes (Records), 2025-04-28)